ПРИКЛАДНОЙ УРОВЕНЬ
Часть III

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Domain Name System (DNS)

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

DNS: система доменных имен

Люди: различные идентификаторы:

- ИНН, имя, номер паспорта

Хосты в Интернете, маршрутизаторы:

- IP-адрес (32-битный) – используется для направления дейтаграмм
- «имя», например, www.yahoo.com – используется людьми
- IP-адрес является иерархическим, и когда мы читаем его слева направо, мы получаем все более определенную информацию о том, в каком месте Интернета располагается хост

Вопрос: как установить соответствие между именем и IP-адресом?

Система DNS:

- распределенная база данных, реализованная с помощью иерархии DNS-серверов
- протокол прикладного уровня: хосты и DNS-сервер взаимодействуют с целью разрешения имен (трансляции имени в адрес)
- работает поверх **UDP** и использует порт 53
  - примечание: функция ядра сети, *реализованная на прикладном уровне*
  - вся сложность на границе сети

DNS: службы, структура

Службы, которые предоставляет DNS:

- трансляция имен хостов в IP-адреса
- поддержка псевдонимов хостов
  - канонические имена и псевдонимы
- поддержка псевдонимов почтовых серверов
- распределение нагрузки и реплицированные веб-серверы: несколько IP-адресов соответствуют одному имени

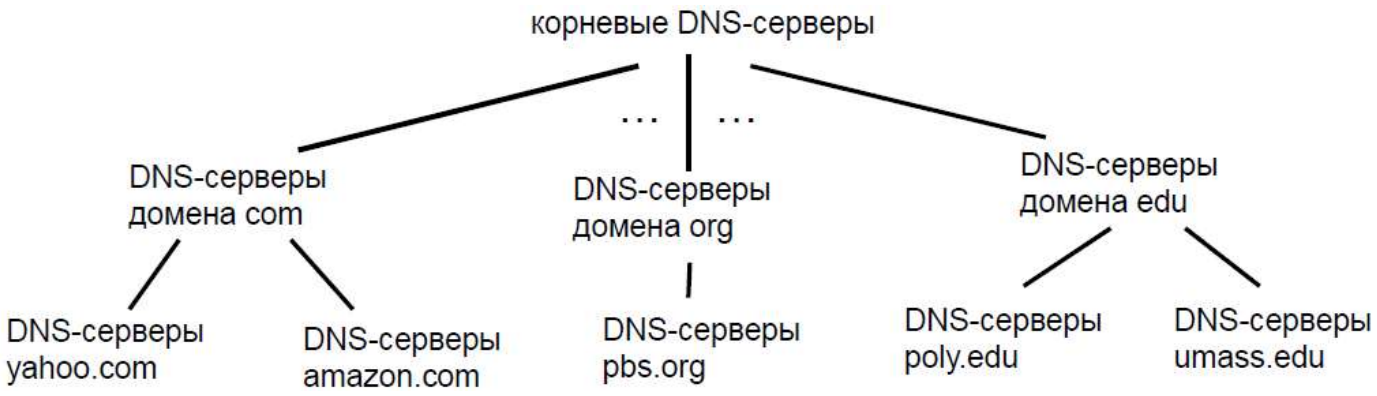
Вопрос: почему бы не централизовать службу DNS?

Проблемы:

- единая точка отказа
- объем трафика
- удаленная центральная база данных
- обслуживание
- нет масштабируемости!

Служба DNS строится как распределенный сервис и является прекрасным примером того, как распределенная база данных может быть реализована в Интернете

DNS: распределенная, иерархическая база данных

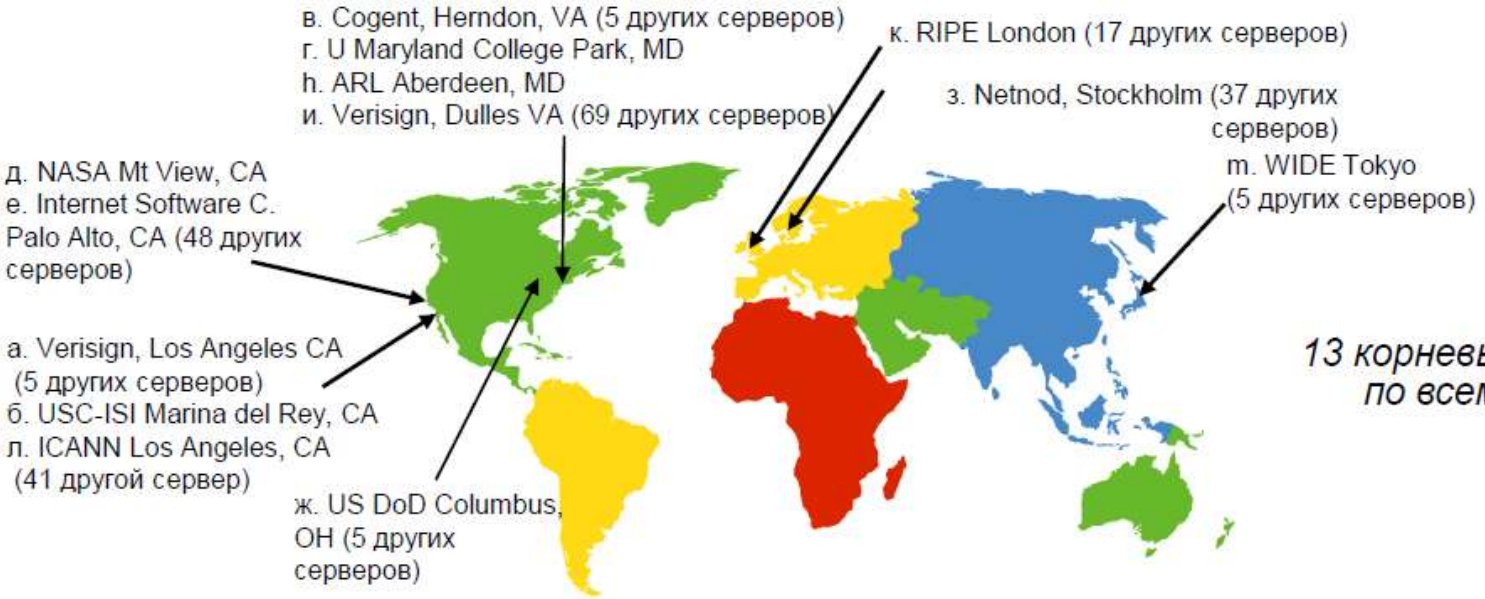


Клиенту нужен IP-адрес для www.amazon.com; приблизительные действия:

- клиент запрашивает корневой сервер и получает адрес DNS-сервера домена com
- клиент запрашивает DNS-сервер домена com и получает адрес DNS для amazon.com
- клиент запрашивает DNS-сервер amazon.com и получает IP-адрес сервера www.amazon.com

DNS: корневые серверы

- локальные DNS-серверы обращаются к ним, если не могут разрешить имя
- корневой DNS-сервер
  - обращается к авторитетным DNS-серверам, если сам не может обработать запрос
  - получает соответствующую пару имя-адрес
  - возвращает пару имя-адрес локальному DNS



13 корневых DNS-серверов по всему миру

Сервера верхнего уровня и авторитетные DNS-серверы

- Сервера верхнего уровня:
  - отвечают за домены com, org, net, edu, aero, jobs, а также за национальные домены: uk, fr, us, jp, ru
  - Network Solutions обслуживает серверы верхнего уровня для домена .com
  - Educause - для домена .edu

- Авторитетные DNS-сервера:
  - собственные DNS-сервера организации, предоставляющие информацию о соответствии имен хостов организации IP-адресам
  - могут обслуживаться самой организацией либо провайдером услуг

Локальный сервер DNS

- Корневые, DNS-серверы верхнего уровня и авторитетные серверы — все принадлежат иерархии DNS-серверов. Но локальный сервер DNS может не принадлежать иерархии

- Каждый Интернет-провайдер предоставляет хотя бы один такой сервер
  - Называется также «сервер имен по умолчанию»

- Когда хост производит запрос DNS, он направляется локальному серверу DNS
  - В кэше локального сервера DNS хранятся пары адрес-имя как результаты недавних запросов (хотя могут быть и неактуальными!)
  - Локальный DNS работает как прокси-сервер, перенаправляя запросы в иерархию DNS

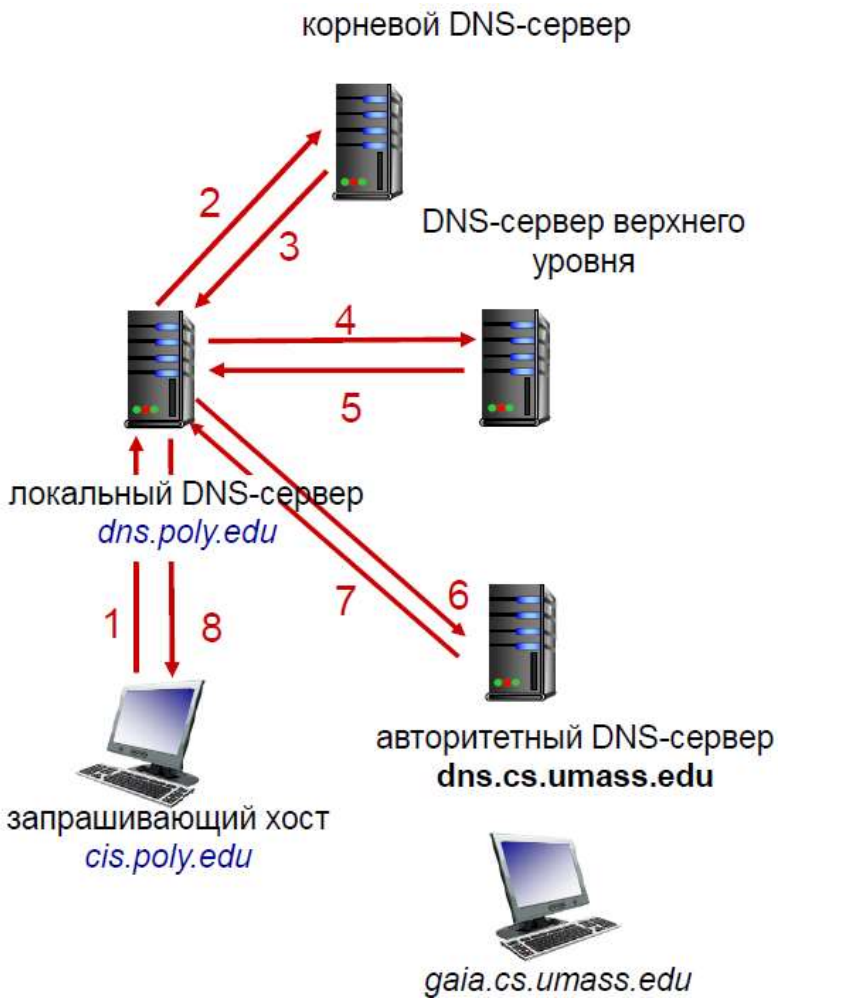
DNS: Пример трансляции имени в адрес

- Хосту cis.poly.edu нужен IP-адрес сервера gaia.cs.umass.edu

Итеративный запрос:

- Сервер, к которому обратились, в ответ дает имена других DNS-серверов
- «Я не знаю ответ, но можете спросить у такого-то сервера»

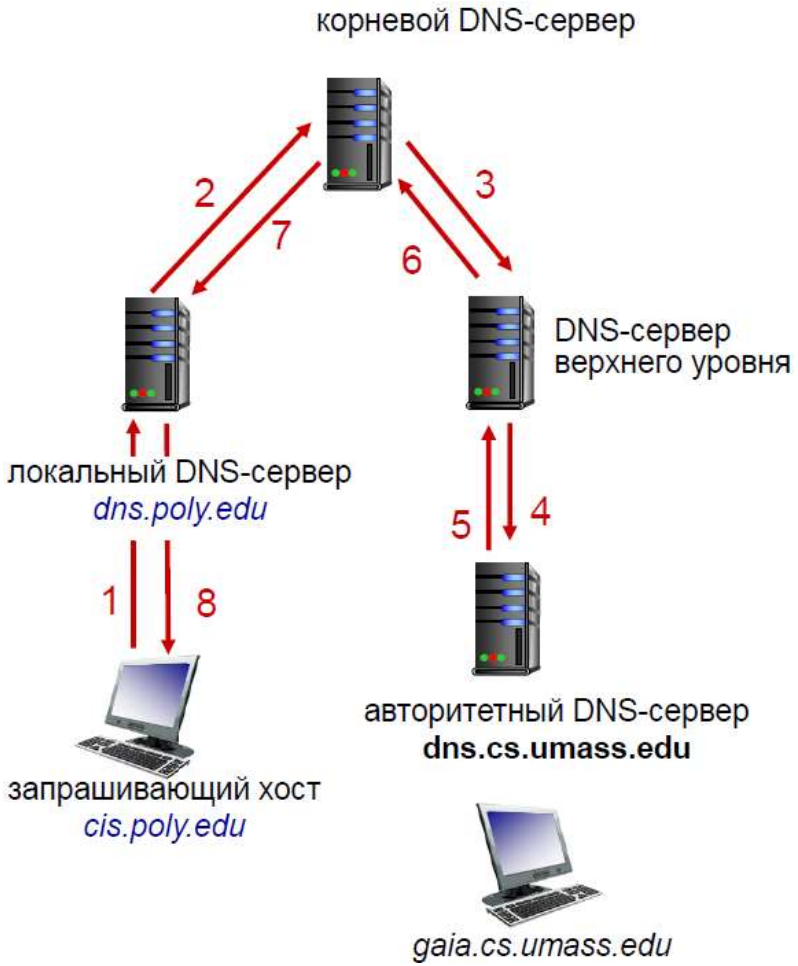
В данном примере для того, чтобы получить таблицу соответствия для одного имени хоста, отправляются **8 DNS-сообщений**:
4 сообщения-запроса и 4 ответных!



DNS: Пример трансляции имени в адрес

Рекурсивный запрос:

- сервер, к которому обратились, берет на себя всю работу по запросам
- серьезная нагрузка на верхние уровни иерархии
- На практике обычные запросы используют схему, представленную на предыдущем слайде: запрос от хоста к локальному DNS-серверу является рекурсивным, а все остальные запросы — итеративными



DNS: кэширование, обновление записей

- Любой DNS-сервер, получив информацию о соответствии имя-адрес, хранит ее в **кэше**
  - информация в кэше периодически сбрасывается (время жизни TTL)
  - в кэше локальных DNS присутствуют данные о серверах верхнего уровня
    - поэтому к корневым серверам можно не обращаться

- Записи в кэше могут быть устаревшими
  - при изменении IP-адреса какого-нибудь хоста, это не будет известно другим хостам в Интернете, пока не пройдет время TTL
  - Eventual consistency

- Механизм обновления DNS регламентируется документом RFC 2136

Записи DNS

DNS: распределенная база данных с ресурсными записями (resource record - RR)

формат ресурсной записи: (имя, значение, тип, ttl)

тип=**A**

- имя – имя хоста
- значение - IP-адрес

тип=**NS**

- имя – имя домена (например, foo.com)
- значение – имя авторитетного DNS-сервера для этого домена

тип=**CNAME**

- имя – псевдоним для реального (канонического) имени
- www.ibm.com – на самом деле имеет имя servereast.backup2.ibm.com
- значение – каноническое имя

тип=**MX**

- значение – каноническое имя почтового сервера для псевдонима в поле Имя.
- С помощью записи MX, компании могут использовать одинаковые псевдонимы для почтовых серверов и для веб-серверов

Сообщения DNS. Заголовок

Запрос и ответ – с одинаковым форматом сообщения

Заголовок сообщения

- идентификатор: 16-битный номер запроса; копируется в ответное сообщение
- флаги:
  - запрос/ответ
  - авторитетный ответ
  - требуется рекурсия
  - рекурсия возможна (в ответном сообщении)

Сообщения DNS

Добавление DNS-записей

- Пример: новый проект *Network Utopia*

- Регистрация имени networkuptopia.com у DNS-регистратора (например, в Network Solutions)
  - предоставление имен, IP-адресов авторитетных DNS-серверов (первичного и вторичного)
  - добавление регистратором двух ресурсных записей в DNS-серверы домена верхнего уровня .com : (networkutopia.com, dns1.networkutopia.com, NS) (dns1.networkutopia.com, 212.212.212.1, A)
  - создание записи типа A авторитетного сервера для www.networkuptopia.com; записи типа MX для networkutopia.com

Атаки на DNS

DDoS-атаки

- бомбардировка корневых серверов огромным числом запросов
  - Неуспешны до сегодняшнего дня
  - Локальные DNS-серверы кэшируют адреса серверов верхнего уровня, позволяя делать запросы в обход корневых DNS
- бомбардировка серверов верхнего уровня
  - потенциально более опасна

Атаки с перенаправлением

- Man-in-middle (человек посередине)
  - Перехват запросов
- Отравление DNS
  - Отправка поддельных пакетов DNS-серверу для кэширования

Использование DNS для DDoS-атаки

- отправка запросов с подменой адреса источника
- «усиление» DNS (размер ответа гораздо больше размера самого запроса)

Одноранговые (P2P) приложения

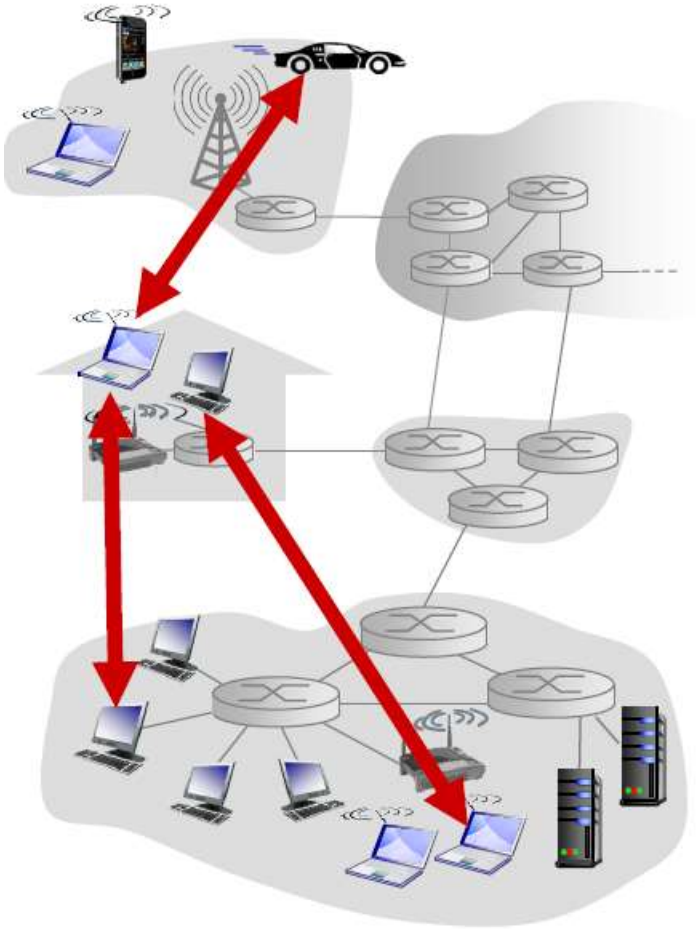
Санкт-Петербургский государственный университет
Факультет математики и компьютерных наук

Одноранговая (P2P) архитектура

- Никакого постоянно работающего сервера
- Произвольные конечные системы взаимодействуют напрямую
- Узлы подключаются периодически и могут менять IP-адреса

Примеры:

- файловый обмен (BitTorrent)
- Потоковое видео (KanKan)
- IP-телефония (Skype)



Файловая раздача: сравнение клиент-серверного и одноранговового вариантов

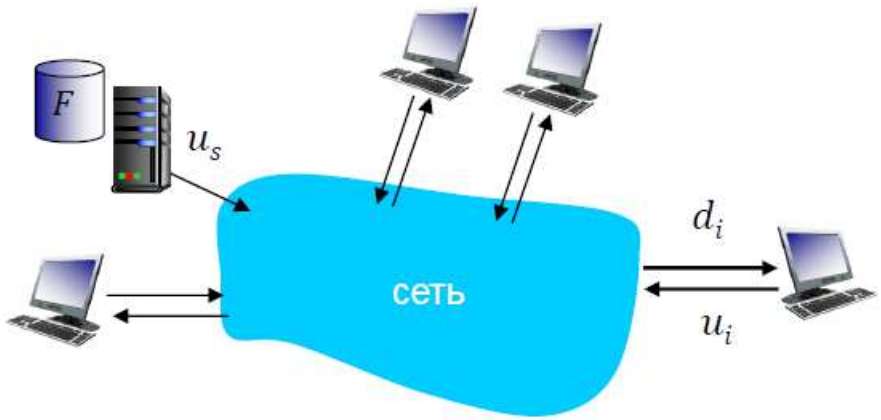
Вопрос: сколько времени уходит на раздачу файла (размером F) от одного сервера к N узлам?

- исходящая/входящая скорость узла ограничены
- временем раздачи будет являться время, которое требуется для того, чтобы получить копию файла на всех **N** узлах

Факультет математики и компьютерных наук

Время раздачи: клиент-серверный вариант

- Сервер: должен последовательно отправить (выгрузить) N копий файла:
  - время на отправку одной копии: F/u_s
  - для N копий: NF/u_s
- Клиент: каждый должен загрузить копию файла
  - d_min = минимальная скорость загрузк среди клиентов
  - минимальное время загрузки среди клиентов:F/d_min



время на раздачу F бит N клиентам при клиент-серверном подходе

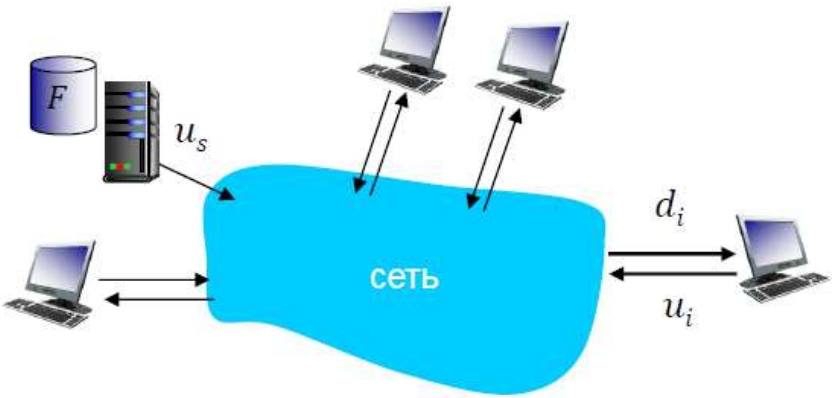
$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

Факультет математики и компьютерных наук

Время раздачи: одноранговый вариант

- сервер: должен выгрузить хотя бы одну копию файла
  - время на отправку одной копии: F/u_s
- клиент: каждый должен загрузить копию файла
  - минимальное время загрузки среди клиентов: F/d_min
- клиенты: должны суммарно загрузить NF бит
  - максимальная скорость выгрузки (ограничивающая максимальную скорость загрузки) равна

$$u_s + \sum u_i$$

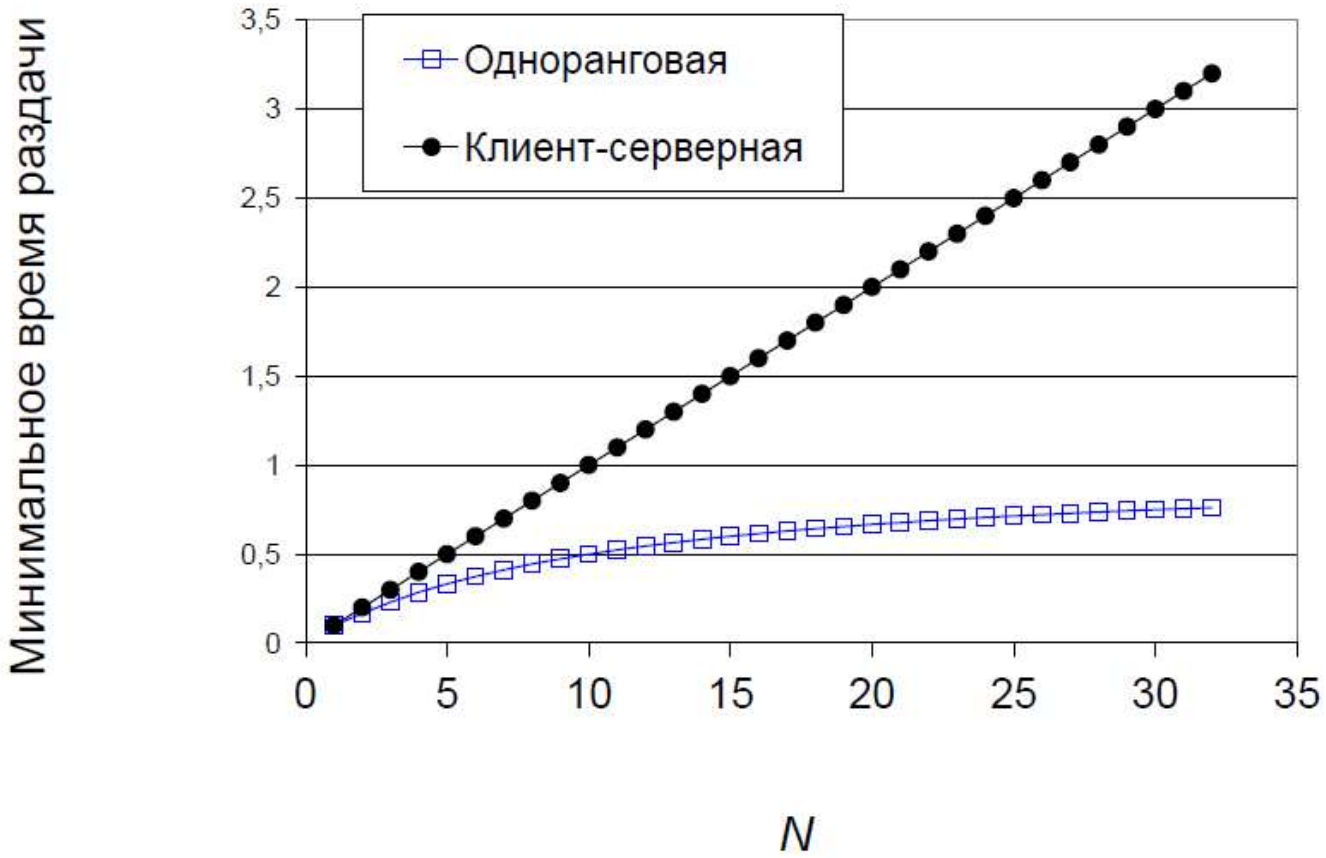


время на раздачу F бит N клиентам при одноранговом подходе $D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$

Факультет математики и компьютерных наук

Сравнение схем раздачи: одноранговая и клиент-серверная

Скорость отдачи клиентом = u, F/u = 1 ч, u_s = 10u, $d_min \geq u_s$

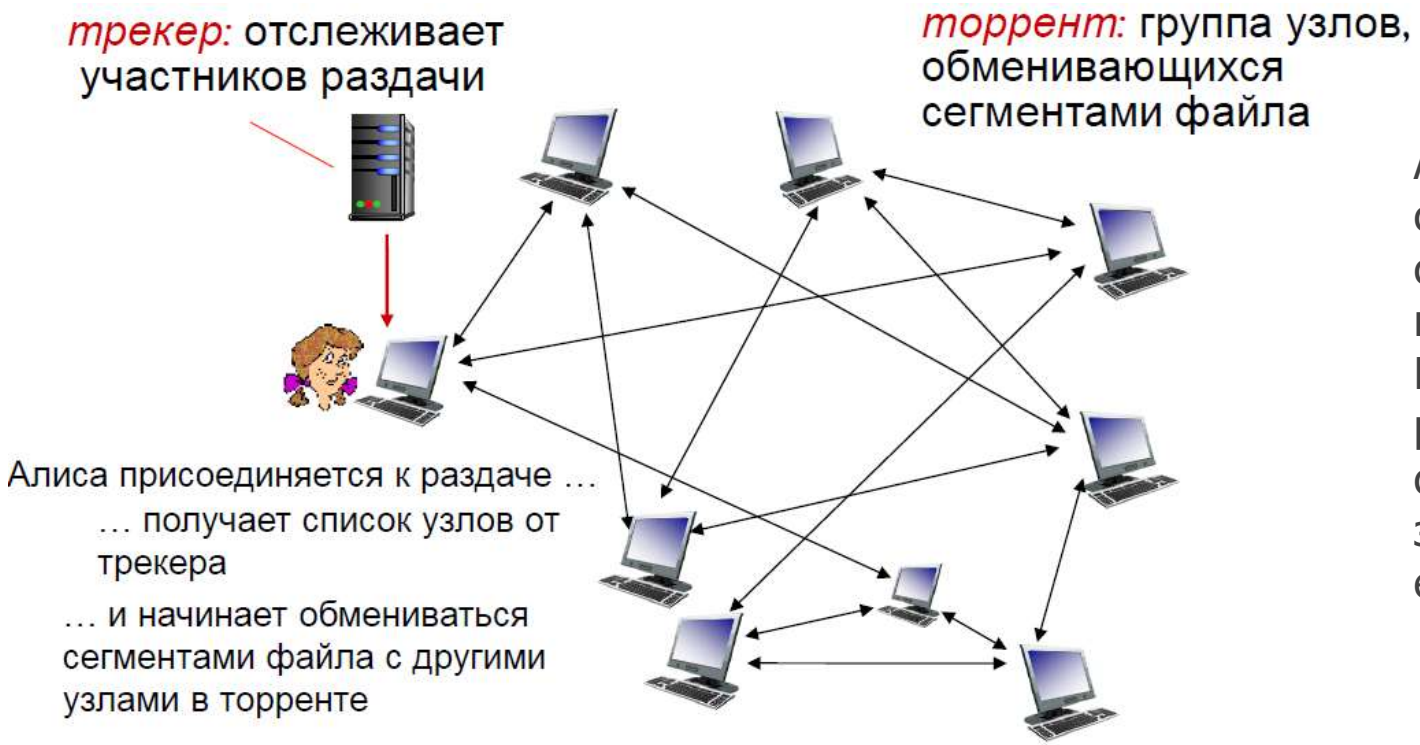


Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Одноранговая раздача файлов: BitTorrent

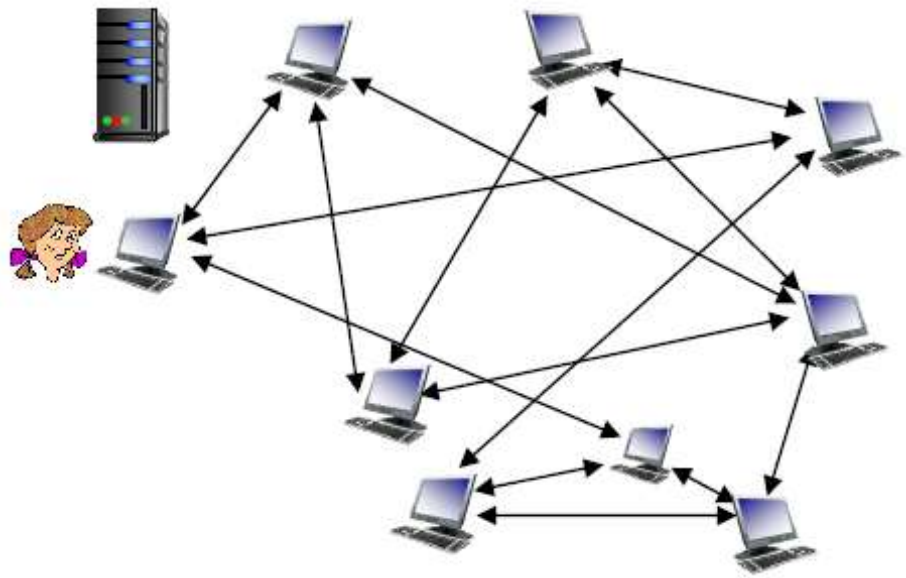
- Набор всех пиров, участвующих в раздаче конкретного файла, называется **торрентом**
- Файл делится на сегменты по 256Кб. Пиры в торренте обмениваются сегментами
- Узлы в торренте получают/отправляют сегменты



Алиса периодически запрашивает у своих соседей (через TCP-соединения) список сегментов, которыми они владеют.
Если у Алисы **L** различных соседей по раздаче, то она получит **L** списков сегментов. Далее Алиса будет запрашивать сегменты, которых у нее еще нет

Одноранговая раздача файлов: BitTorrent

- Присоединяющийся к торренту узел (пир):
  - не имеет сегментов, но начинает получать их от других узлов
  - регистрируется трекером и получает список узлов, соединяется с набором узлов («соседями»)
- Узлы одновременно загружают и отдают сегменты
- Узел может менять партнеров по обмену сегментами
- Отток: узлы появляются и исчезают
- При получении всего файла узел может (эгоистично) покинуть торрент или остаться в раздаче (бескорыстно)



BitTorrent: запрос и отправка сегментов

Запрос сегментов:

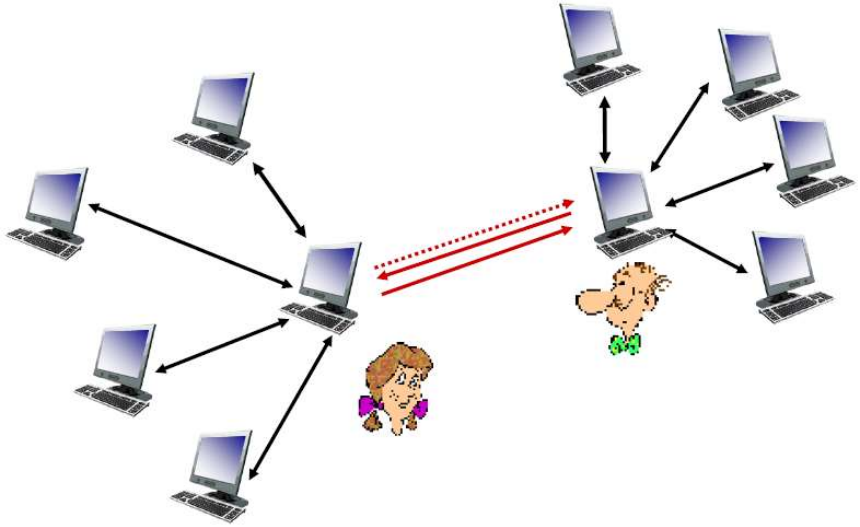
- в любой момент времени различные узлы хранят разный набор сегментов
- Алиса периодически запрашивает у каждого узла список сегментов, которые у него есть
- Алиса запрашивает отсутствующие у нее сегменты по принципу «**сначала редкие**». Тогда редкие сегменты начинают раздаваться быстрее, и это приводит к тому, что число копий каждого сегмента в торренте приблизительно уравнивается

- Отправка: ты-мне-я-тебе
- Алиса отправляет сегменты четырем узлам, которые снабжают ее на максимальной скорости
  - другие узлы Алиса блокирует (не получает от них сегменты)
  - каждые 10 с четверка топ-узлов пересматривается
- каждые 30 с: случайно выбирается новый узел и начинается отправка ему сегментов
  - это называется «оптимистическая разблокировка» узла
  - новый узел присоединяется к топ-четверке, остальные «удушаются»

BitTorrent: ты-мне-я-тебе

(1) Алиса оптимистически разблокирует Боба
(2) Алиса становится одним из четырех лучших раздающих для Боба;
Боб отвечает взаимностью
(3) Боб становится одним из четырех лучших раздающих для Алисы

Как увеличить скорость загрузки: найдешь лучшего раздающего партнера - получишь файл быстрее!



★ Распределенная хеш-таблица (DHT)

- DHT: распределенная база данных одноранговой сети
- база данных содержит пары (ключ, значение); Примеры:
  - ключ: ИНН; значение: имя владельца
  - ключ: название видеофайла; значение: IP-адрес хоста
- пары (ключ, значение) распределены среди миллионов узлов
- узел делает запрос к таблице с помощью ключа
  - DHT возвращает значение, соответствующее ключу
- узлы добавляют пары (ключ, значение) в таблицу
- каждый узел (пир) в такой системе будет хранить только небольшую часть всех пар ключ, значение

Как назначать ключи узлам?

- Главная задача:
  - присваивание пар (ключ, значение) узлам

  Ключ – это не обязательно целое число. Поэтому сперва его нужно преобразовать в такое целое число с помощью хеш-функции

- Основная идея:
  - преобразовать ключ в целое число
  - каждому узлу присвоить целочисленный идентификатор
  - сохранить пару (ключ, значение) в узле с ближайшим к ключу идентификатором

Идентификаторы в хеш-таблицах

- Каждому узлу присваивается целочисленный идентификатор в диапазоне [0, 2^n−1]
  - идентификатор выражается n-битным представлением

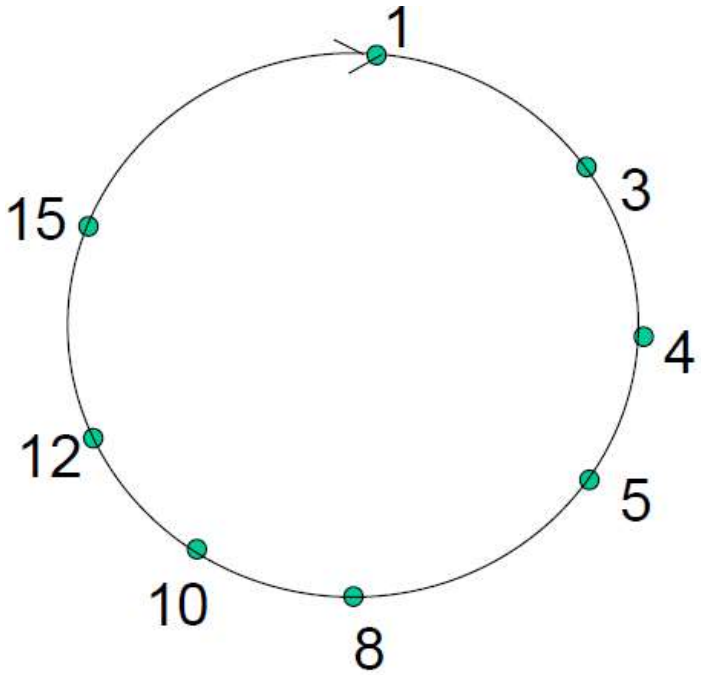
- Каждый ключ – целое число в том же диапазоне
- Используем хеш-функцию для получения целочисленных из исходных ключей
  - ключ = хеш
  - поэтому используем слово «хеш» в термине хеш-таблица

Присваивание ключей узлам

Учитывая, что каждый пир имеет целочисленный идентификатор и каждый ключ является целым числом в том же диапазоне, естественным подходом было бы присвоить каждую пару ключ-значение пиру, имеющему ближайший идентификатор к ключу

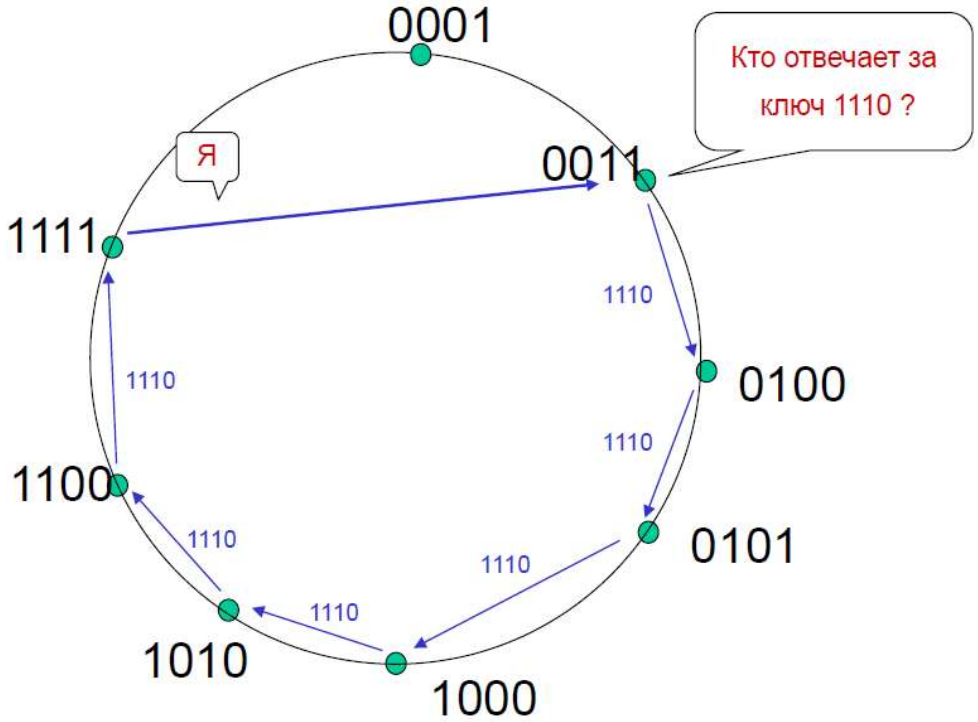
- Правило: присваивать ключ узлу с ближайшим идентификатором
- Договоренность: ближайший – непосредственный последователь (ближайший следующий)
- Пример n=4; узлы: 1, 3, 4, 5, 8, 10, 12, 14;
  - ключ = 13, последователь = 14
  - ключ = 15, последователь = 1

Циркулярные хеш-таблицы



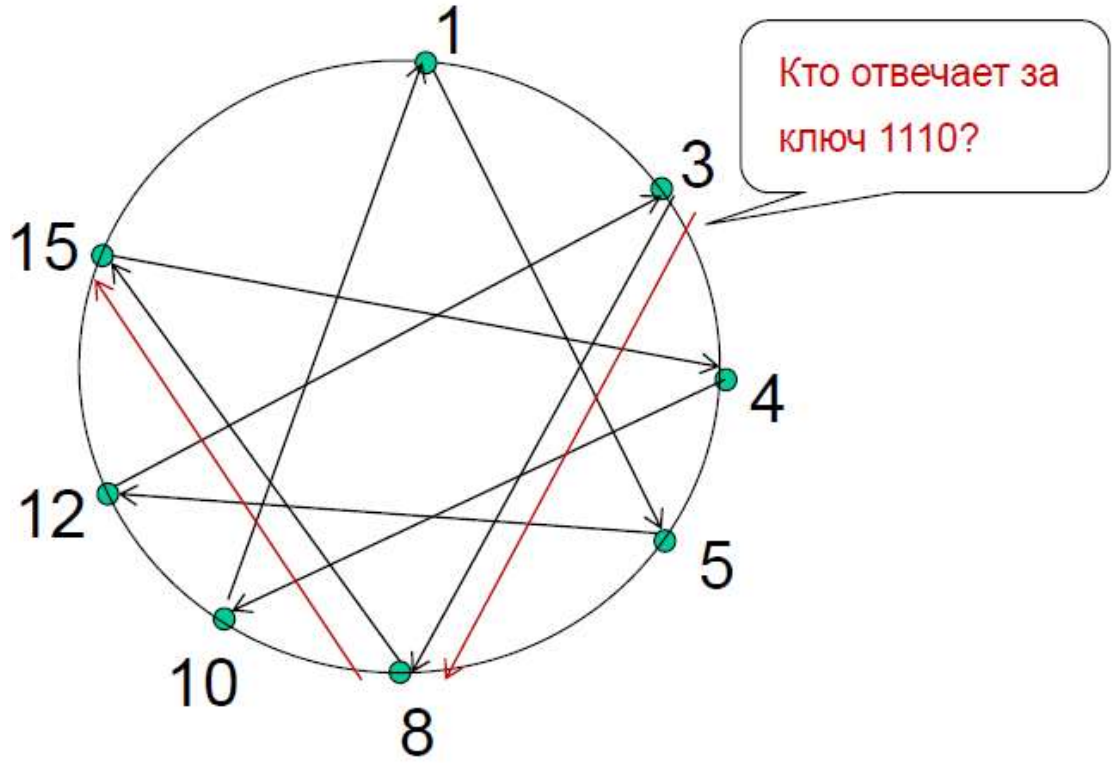
- Как определить узел, идентификатор которого является ближайшим к ключу?
- Каждый узел заботится только о непосредственных последователе и предшественнике
- Круговое размещение пиров — это особый случай оверлейной (наложенной) сети. Каналами связи в оверлейных сетях являются не физические, а виртуальные

Циркулярные хеш-таблицы



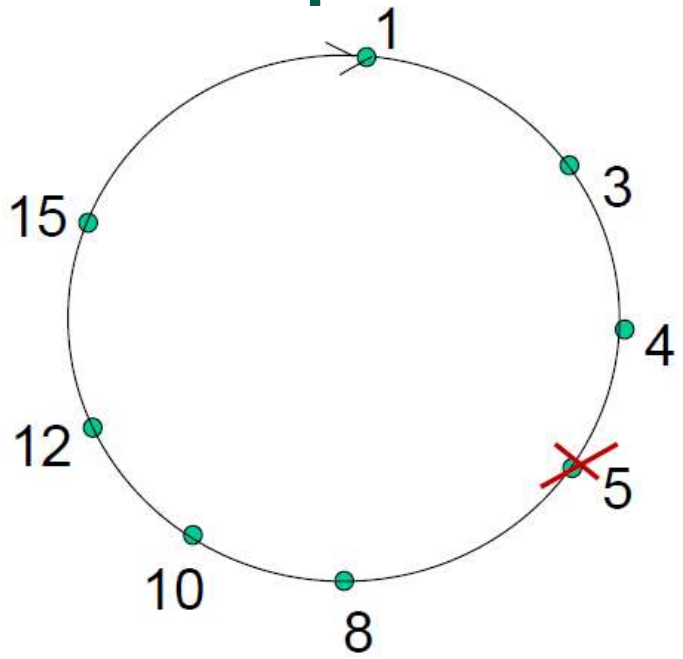
- Чтобы найти узел, ответственный за ключ (в худшем случае), все N узлов хеш-таблицы будут вынуждены перенаправлять сообщения по кругу, и в среднем будет отправлено N/2 сообщений
- Ближайший - это непосредственный последователь

Циркулярная хеш-таблица со ссылками



- каждый узел отслеживает последователя, предшественника и соседа, на которого указывает ссылка
- число сообщений уменьшилось с 6 до 2
- возможный вариант построения таблицы: $O(\log N)$ соседей и $O(\log N)$ сообщений на запрос

Отток пиров



Обработка оттока пиров:

- узлы появляются и исчезают (происходит отток)
- каждый узел отслеживает двух своих последователей
- каждый узел периодически отправляет эхо-запрос двум последователям, проверяя «живы» ли они
- при исчезновении первого последователя второй становится на его место

Пример: пир 5 исчезает

- пир 4 обнаруживает, что 5-й пропал; его действия: делает 8-го своим первым последователем; спрашивает у 8-го о его первом последователе (10); делает 10-го своим вторым последователем
- что будет когда пир 13 захочет присоединиться к таблице?

Распределенные хеш-таблицы в BitTorrent

BitTorrent использует **Kademlia DHT** для создания распределенного трекера.

Ключом является идентификатор торрента, а значением — IP-адреса всех пиров, участвующих в торренте